



Imports

**** Import pandas, numpy, matplotlib, and seaborn. Then set %matplotlib inline (You'll import sklearn as you need it.)****

```
In [1]: import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
```

Get the Data

We'll work with the Ecommerce Customers csv file from the company. It has Customer info, such as Email, Address, and their color Avatar. Then it also has numerical value columns:

- Avg. Session Length: Average session of in-store style advice sessions.
- Time on App: Average time spent on App in minutes
- Time on Website: Average time spent on Website in minutes
- Length of Membership: How many years the customer has been a member.

**** Read in the Ecommerce Customers csv file as a DataFrame called customers.****

```
In [2]: df = pd.read_csv('Ecommerce Customers')
```

Check the head of customers, and check out its info() and describe() methods.

```
In [3]: df.head()
```

Out[3]:

	Email	Address	Avatar	Avg. Session Length	Time
0	mstephenson@fernandez.com	835 Frank Tunnel\ nWrightmouth, MI 82180-9605	Violet	34.497268	12.65
1	hduke@hotmail.com	4547 Archer Common\ nDiazchester, CA 06566-8576	DarkGreen	31.926272	11.10
2	pallen@yahoo.com	24645 Valerie Unions Suite 582\ nCobbborough, D...	Bisque	33.000915	11.33
3	riverarebecca@gmail.com	1414 David Throughway\ nPort Jason, OH 22070-1220	SaddleBrown	34.305557	13.71
4	mstephens@davidson-herman.com	14023 Rodriguez Passage\ nPort Jacobville, PR 3...	MediumAquaMarine	33.330673	12.79

In [4]: `df.describe()`

Out[4]:

	Avg. Session Length	Time on App	Time on Website	Length of Membership	Yearly Amount Spent
count	500.000000	500.000000	500.000000	500.000000	500.000000
mean	33.053194	12.052488	37.060445	3.533462	499.314038
std	0.992563	0.994216	1.010489	0.999278	79.314782
min	29.532429	8.508152	33.913847	0.269901	256.670582
25%	32.341822	11.388153	36.349257	2.930450	445.038277
50%	33.082008	11.983231	37.069367	3.533975	498.887875
75%	33.711985	12.753850	37.716432	4.126502	549.313828
max	36.139662	15.126994	40.005182	6.922689	765.518462

In [6]: `df.info()`

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 500 entries, 0 to 499
Data columns (total 8 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Email                                500 non-null    object
1   Address                             500 non-null    object
2   Avatar                              500 non-null    object
3   Avg. Session Length                 500 non-null    float64
4   Time on App                         500 non-null    float64
5   Time on Website                     500 non-null    float64
6   Length of Membership                 500 non-null    float64
7   Yearly Amount Spent                 500 non-null    float64
dtypes: float64(5), object(3)
memory usage: 31.4+ KB

```

Exploratory Data Analysis

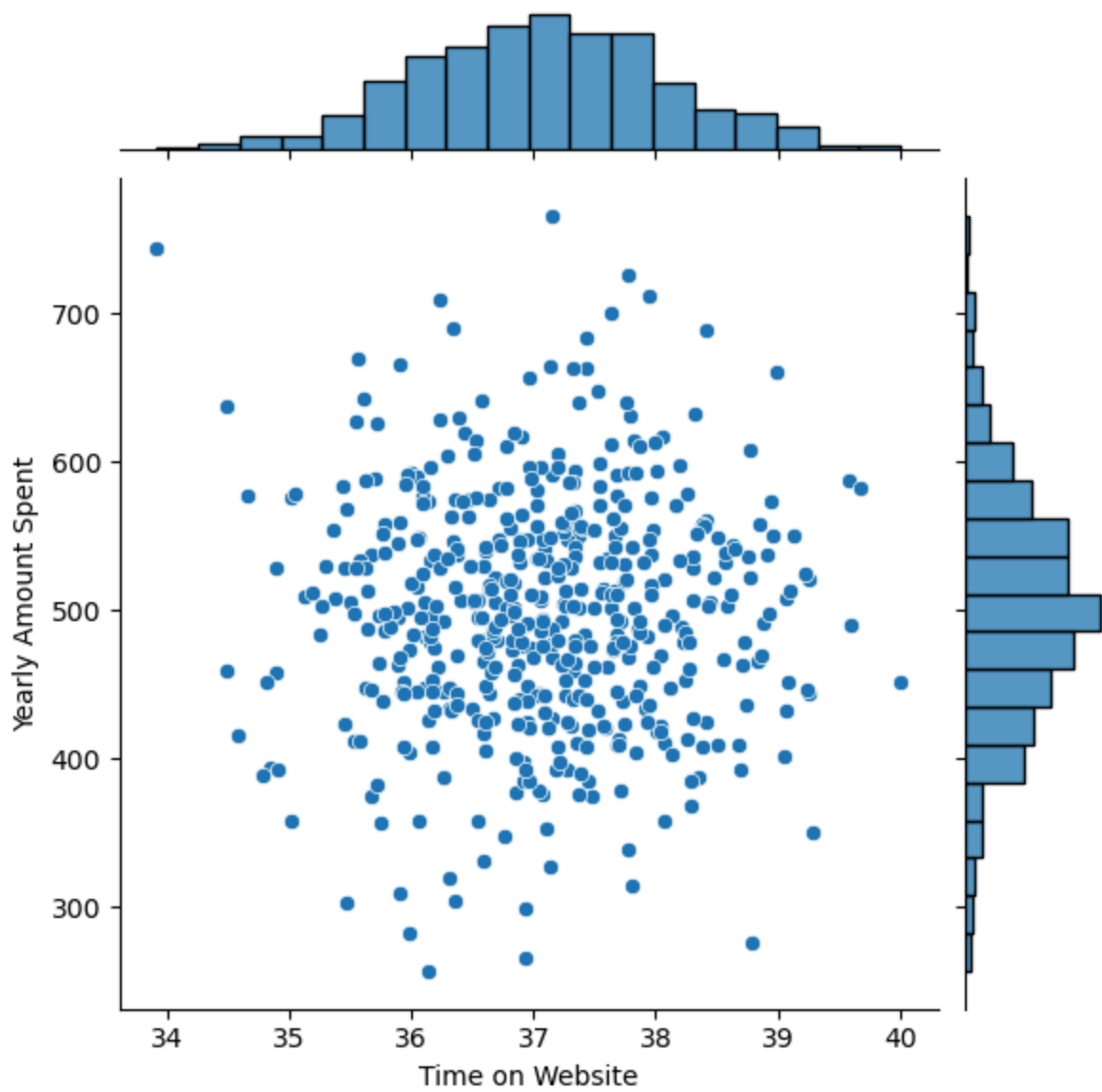
Let's explore the data!

For the rest of the exercise we'll only be using the numerical data of the csv file.

Use seaborn to create a jointplot to compare the Time on Website and Yearly Amount Spent columns. Does the correlation make sense?

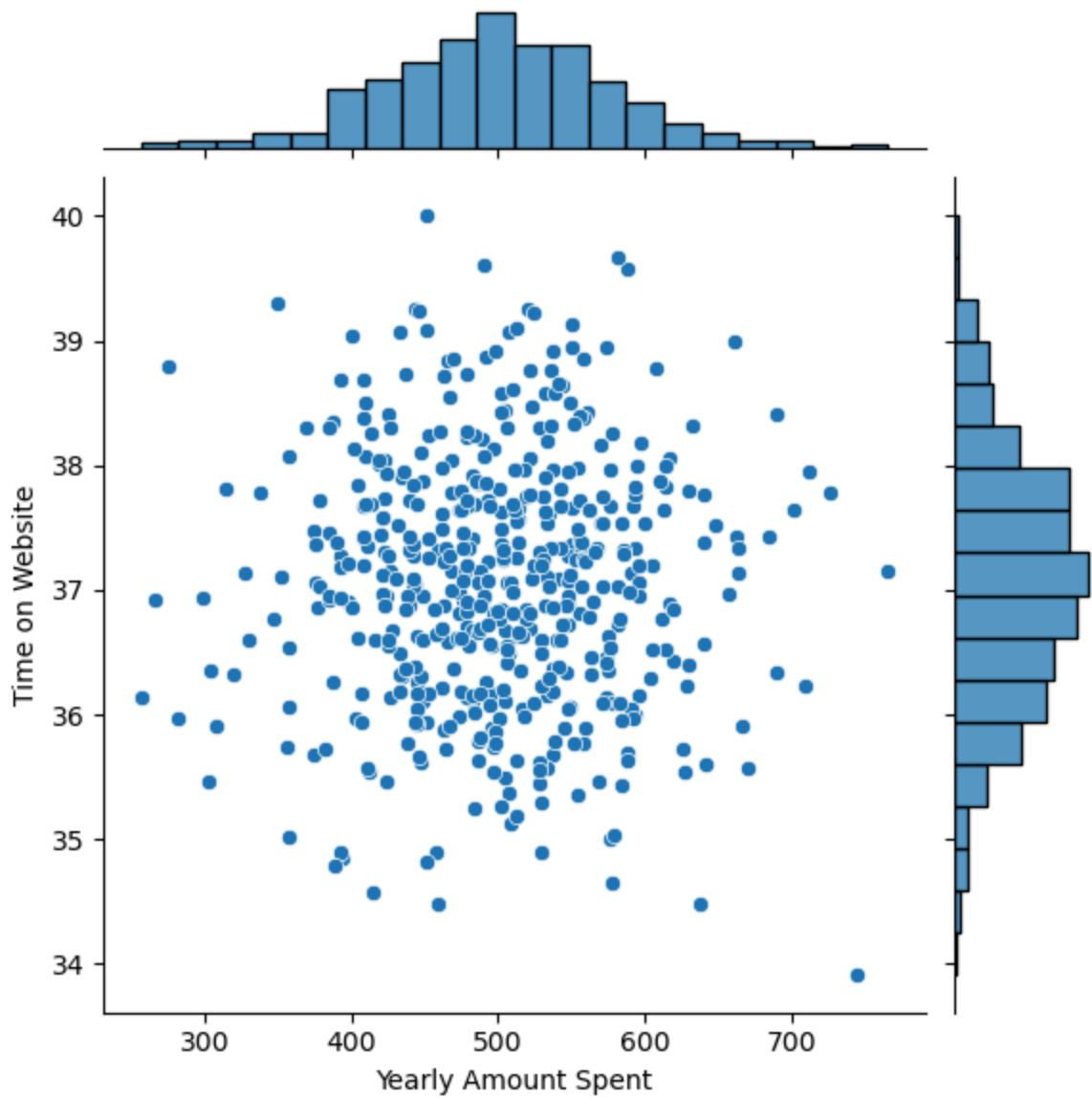
```
In [8]: sns.jointplot(x = df['Time on Website'], y = df['Yearly Amount Spent'])
```

```
Out[8]: <seaborn.axisgrid.JointGrid at 0x1f6a9290b90>
```



```
In [9]: sns.jointplot(x = df['Yearly Amount Spent'], y = df['Time on Website'])
```

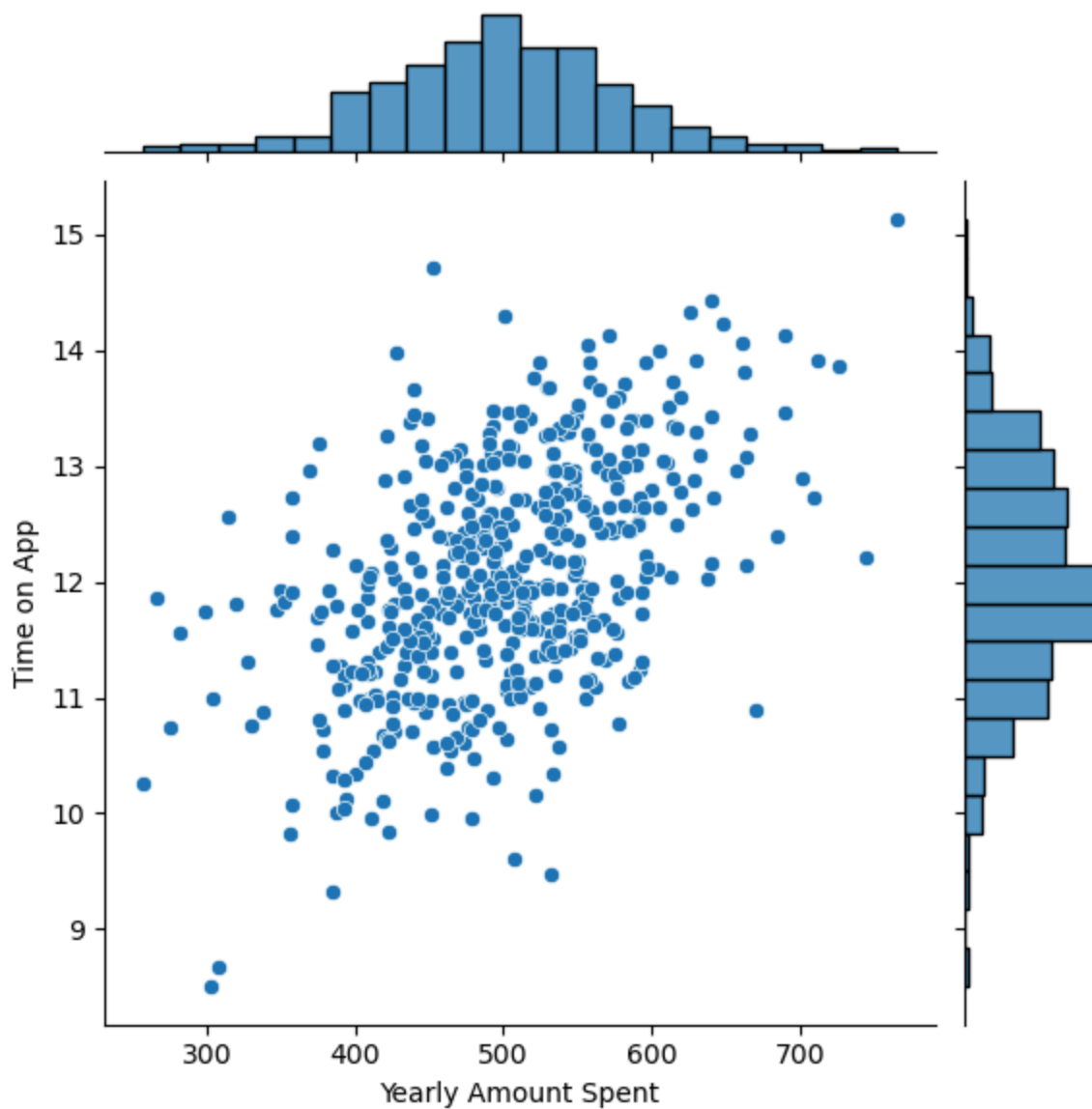
```
Out[9]: <seaborn.axisgrid.JointGrid at 0x1f6a89c7cb0>
```



**** Do the same but with the Time on App column instead. ****

```
In [10]: sns.jointplot(x = df['Yearly Amount Spent'], y = df['Time on App'])
```

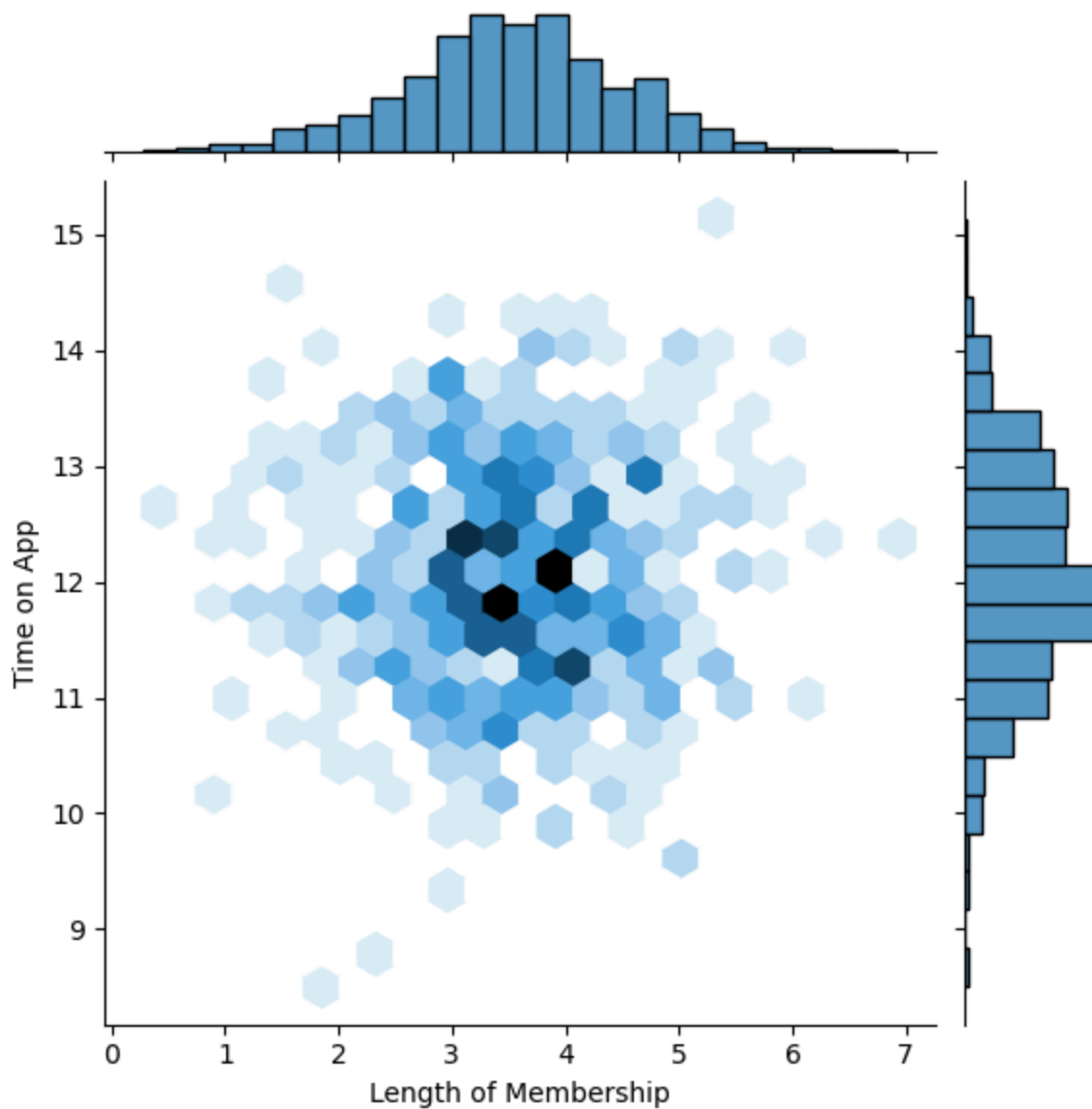
```
Out[10]: <seaborn.axisgrid.JointGrid at 0x1f6a996d190>
```



**** Use jointplot to create a 2D hex bin plot comparing Time on App and Length of Membership.****

```
In [11]: sns.jointplot(x = df['Length of Membership'], y = df['Time on App'], kind = 'h
```

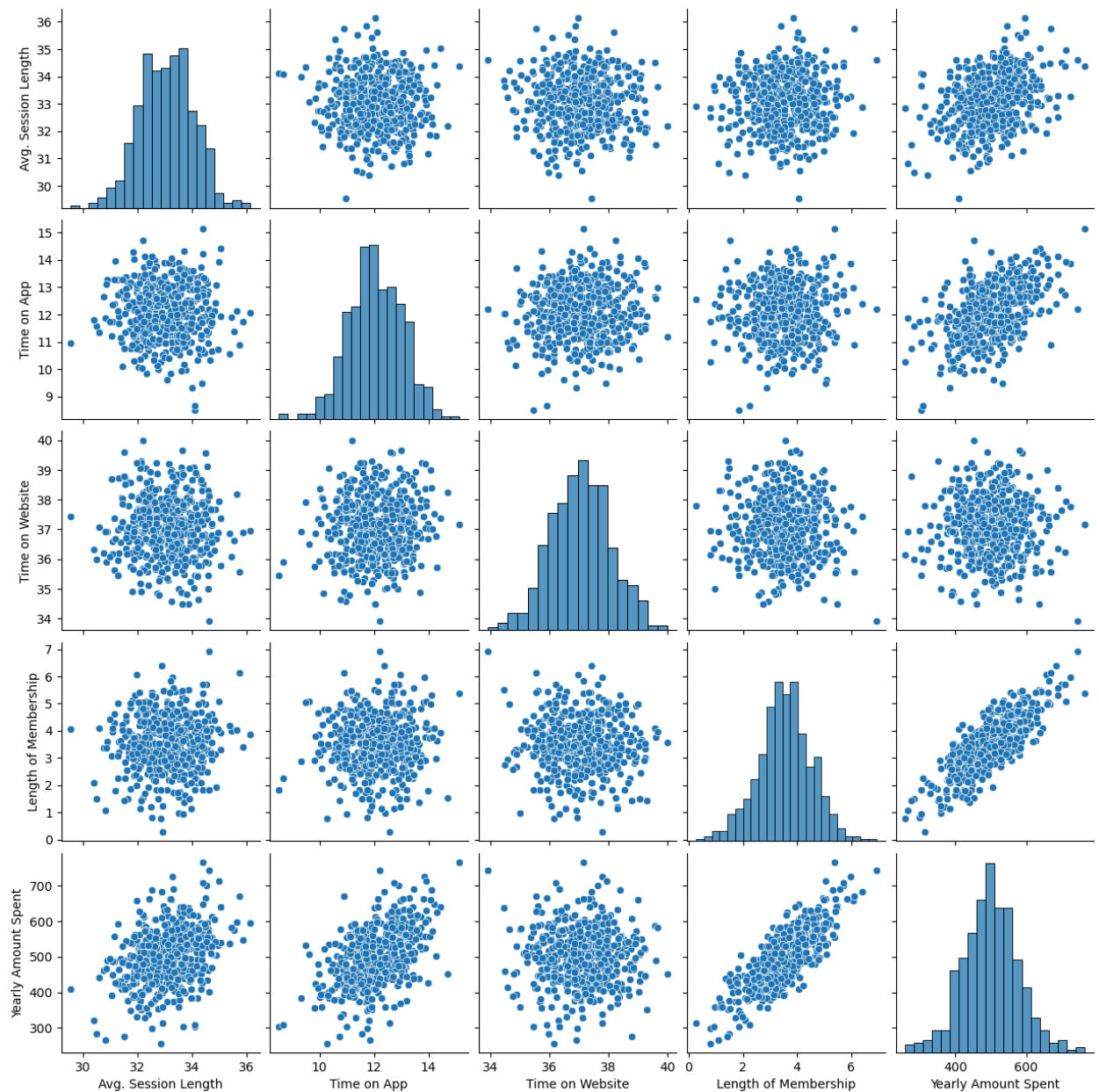
```
Out[11]: <seaborn.axisgrid.JointGrid at 0x1f6a929c080>
```



Let's explore these types of relationships across the entire data set. Use **pairplot** to recreate the plot below.(Don't worry about the the colors)

```
In [12]: sns.pairplot(df)
```

```
Out[12]: <seaborn.axisgrid.PairGrid at 0x1f6a7db8410>
```



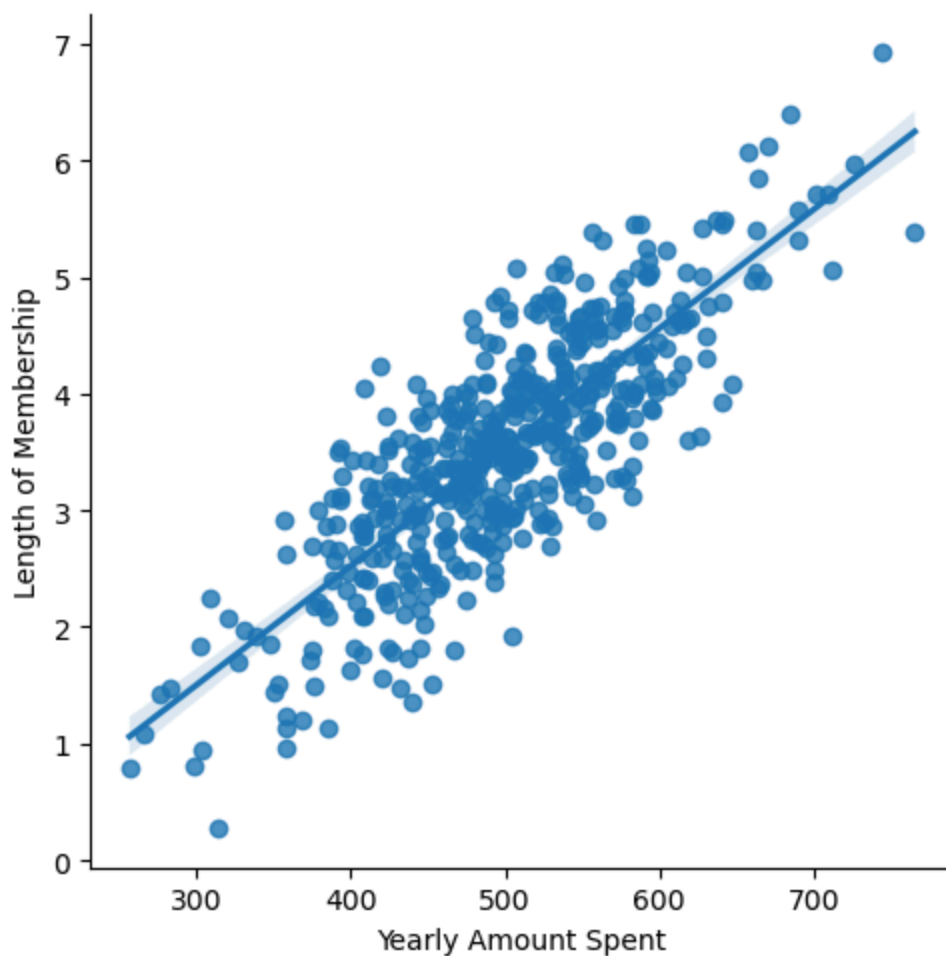
Based off this plot what looks to be the most correlated feature with Yearly Amount Spent?

The most Correlated feature with "Yearly Amount Spent" is "Length of Membership".

****Create a linear model plot (using seaborn's lmlplot) of Yearly Amount Spent vs. Length of Membership. ****

```
In [17]: sns.lmlplot(data = df, x = 'Yearly Amount Spent', y = 'Length of Membership')
```

```
Out[17]: <seaborn.axisgrid.FacetGrid at 0x1f6ac59d550>
```

Training and Testing Data

Now that we've explored the data a bit, let's go ahead and split the data into training and testing sets. ** Set a variable X equal to the numerical features of the customers and a variable y equal to the "Yearly Amount Spent" column. **

```
In [20]: X = df[['Avg. Session Length', 'Time on App', 'Time on Website', 'Length of Membership']]
         y = df['Yearly Amount Spent']
```

In []:

** Use `model_selection.train_test_split` from `sklearn` to split the data into training and testing sets. Set `test_size=0.3` and `random_state=101`**

```
In [21]: from sklearn.model_selection import train_test_split
```

```
In [22]: X_test, X_train, y_test, y_train = train_test_split(X, y, test_size = 0.3, random_state=101)
```

Training the Model

Now its time to train our model on our training data!

**** Import LinearRegression from sklearn.linear_model ****

```
In [23]: from sklearn.linear_model import LinearRegression
```

Create an instance of a LinearRegression() model named lm.

```
In [24]: lm = LinearRegression()
```

**** Train/fit lm on the training data.****

```
In [25]: lm.fit(X_test,y_test)
```

```
Out[25]: 

LinearRegression ⓘ ?  
LinearRegression()


```

Print out the coefficients of the model

```
In [30]: coefficients = pd.DataFrame(lm.coef_,X.columns,columns=['Coefficient'])  
coefficients
```

```
Out[30]:
```

	Coefficient
Avg. Session Length	25.981550
Time on App	38.590159
Time on Website	0.190405
Length of Membership	61.279097

Predicting Test Data

Now that we have fit our model, let's evaluate its performance by predicting off the test values!

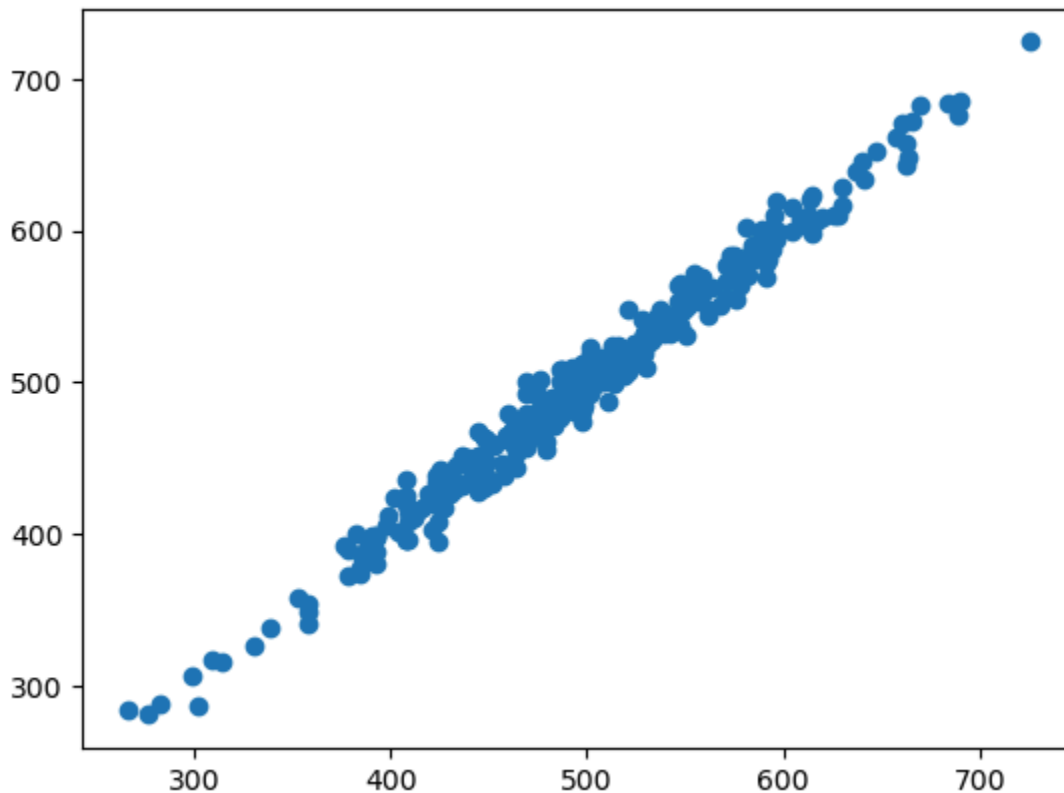
**** Use lm.predict() to predict off the X_test set of the data.****

```
In [31]: prediction = lm.predict(X_test)
```

**** Create a scatterplot of the real test values versus the predicted values. ****

```
In [32]: plt.scatter(y_test, prediction)
```

```
Out[32]: <matplotlib.collections.PathCollection at 0x1f6b2760c80>
```



Evaluating the Model

Let's evaluate our model performance by calculating the residual sum of squares and the explained variance score (R^2).

**** Calculate the Mean Absolute Error, Mean Squared Error, and the Root Mean Squared Error. Refer to the lecture or to Wikipedia for the formulas****

```
In [34]: from sklearn import metrics
```

```
In [40]: print('Mean Squared Error:', metrics.mean_squared_error(y_test , prediction))
print('Root Mean Squared Error:', np.sqrt(metrics.mean_squared_error(y_test ,
print('Mean Absolute Error:', metrics.mean_absolute_error(y_test , prediction)
```

```
Mean Squared Error: 106.85137432285721
Root Mean Squared Error: 10.336893843068005
Mean Absolute Error: 8.181347287342842
```

Residuals

You should have gotten a very good model with a good fit. Let's quickly explore the residuals to make sure everything was okay with our data.

Plot a histogram of the residuals and make sure it looks normally distributed. Use either seaborn distplot, or just plt.hist().

```
In [42]: sns.distplot(df['Yearly Amount Spent'])
```

C:\Users\rezay\AppData\Local\Temp\ipykernel_14256\429359120.py:1: UserWarning:

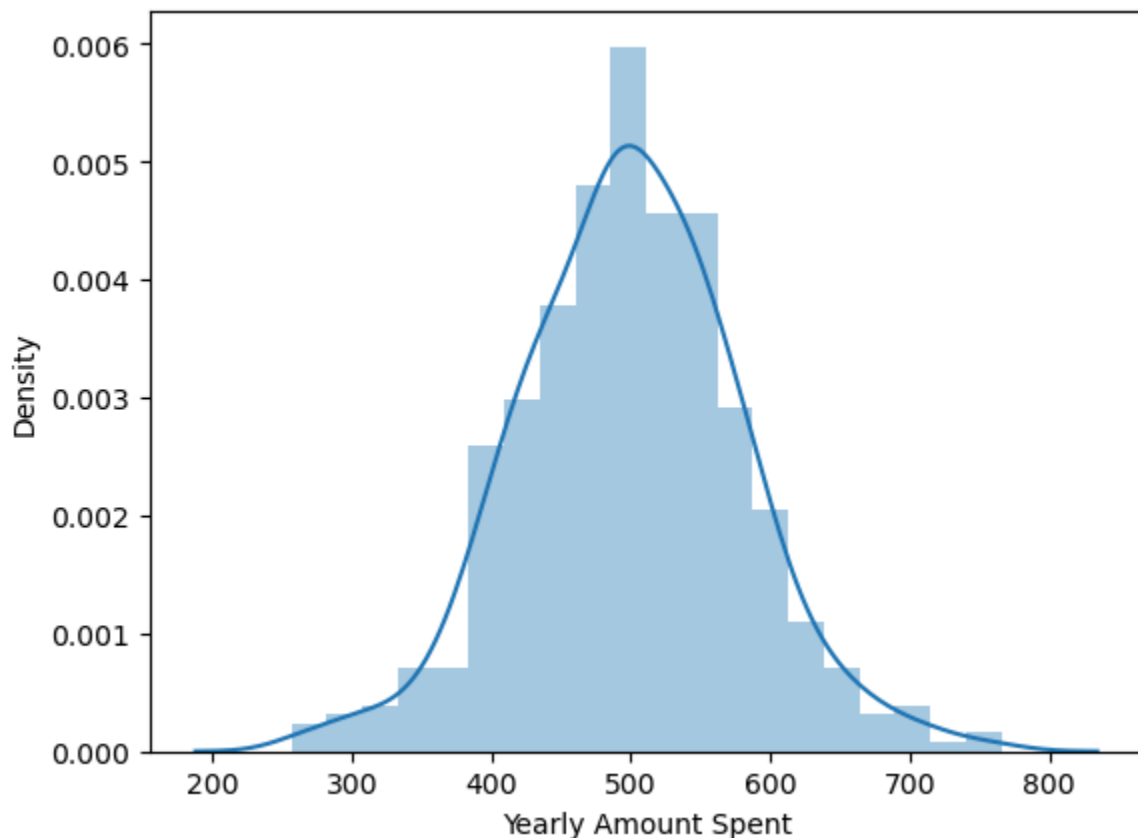
`distplot` is a deprecated function and will be removed in seaborn v0.14.0.

Please adapt your code to use either `displot` (a figure-level function with similar flexibility) or `histplot` (an axes-level function for histograms).

For a guide to updating your code to use the new functions, please see <https://gist.github.com/mwaskom/de44147ed2974457ad6372750bbe5751>

```
sns.distplot(df['Yearly Amount Spent'])
```

```
Out[42]: <Axes: xlabel='Yearly Amount Spent', ylabel='Density'>
```



Conclusion

We still want to figure out the answer to the original question, do we focus our effort on mobile app or website development? Or maybe that doesn't even really matter, and Membership Time is what is really important. Let's see if we can interpret the coefficients at all to get an idea.

**** Recreate the dataframe below. ****

```
In [43]: pd.DataFrame(lm.coef_, X.columns, columns = ['Coefficient'])
```

```
Out[43]:
```

	Coefficient
Avg. Session Length	25.981550
Time on App	38.590159
Time on Website	0.190405
Length of Membership	61.279097

**** How can you interpret these coefficients? ****

1 unit change in 'Yearly Amount Spent' shall lead to this much(Coefficient) change in their features or attributes.

Do you think the company should focus more on their mobile app or on their website?

Company should focus more on App as it's coefficient of 'Time on App' change more if they spending on this feature or attribute

Great Job!

Congrats on your contract work! The company loved the insights! Let's move on.